

Session 1 — Study Guide: Routing in B4F Hub

This guide explains, in depth, what changed in B4F Hub in Session 1 and, more importantly, **why**. If you forgot most of the class, you should be able to read this alone and rebuild your understanding. The short version is in the Student Notes; this is the long version.

Your team's teamwork exercise was the opportunity detail page (sections 9–13 below) — everything in this guide, including deleting the old hand-rolled history code in section 13, was reached and taught.

1. Where the session starts and ends

Start. B4F Hub is two separate projects: `client/` (React + TypeScript + Vite) and `server/` (Node + Express, port 3001). You run them in two terminals. The client is one page: the Community panel and the Opportunities panel are always on screen together, and the address is always the same.

End. The client is a multi-page app. Home, Community and Opportunities are pages with their own addresses (`/`, `/community`, `/opportunities`), reached through a Navbar that highlights where you are, and your team built a real detail page for each opportunity as the teamwork exercise: each opportunity now has its own address (`/opportunities/101`, `/opportunities/205`, ...) with a full detail page and a real Back button. The server did not change at all today.

2. The problem: one page, one address

A familiar idea first. Think of a book. Every page has a number. You can say "open the book on page 120", and someone else can go straight to it. Our app was a book with one giant page: however far you scrolled or clicked, it was always "page 1".

What was wrong with that?

- If you wanted to send a teammate a link to the Opportunities, there was nothing to send.
- If you refreshed, you lost "where you were".
- The browser's Back button did nothing useful, because the browser thought you never went anywhere.
- We had even written code to **fake** the missing history: a hand-made stack of "previously viewed opportunities" in `OpportunitiesSection`. That code only existed because the app had no real navigation.

The idea of this whole session: **make the address in the browser part of the application**. The address says what is on screen, and the app changes what it shows when the address changes.

TEXT

```
address (URL) -> which page component is shown
/              -> HomePage
/community     -> CommunityPage
/opportunities -> OpportunitiesPage
```

3. What is React Router, and why do we install it?

React by itself knows nothing about addresses. **React Router** is a library that connects the browser's address bar to your components. It is a frontend tool, so it is installed in `client/` only:

BASH

```
cd client
npm install "react-router-dom@^7.18.4"
```

The `server/` project is never touched: it does not know or care that the client now has pages.

4. `BrowserRouter` : switching on address awareness

Every router feature needs to live inside a *router*. `BrowserRouter` is that router: it watches the browser's real address and tells everything inside it when the address changes. We wrap the whole app once, in `main.tsx`:

TSX

```
<StrictMode>
  <BrowserRouter>
    <App />
  </BrowserRouter>
</StrictMode>,
```

Why at the very top? Because any component, anywhere in the app, may want to use routing. **Why only once?** One app, one address bar.

Common mistake. Using `Routes` or `Link` outside `BrowserRouter`. React Router throws an error as soon as the component renders, because there is no router to talk to.

5. Pages, `Routes` and `Route`

Familiar idea. You already know that a component is a function that returns JSX. A **page** is just a component that an *address* shows. We keep pages in their own folder, `src/pages/`, so we can see at a glance which components are route targets. A **section** (like `CommunitySection`) is a reusable piece of UI. That difference matters: we did not rewrite `Community`; we wrapped it in a thin page.

TSX

```
// Thin page wrapper: the real work still lives in CommunitySection, which
// hasn't changed at all. This file's only job is to be the thing a <Route>
// renders at /community.
function CommunityPage({ onNotify }: CommunityPageProps) {
  return <CommunitySection onNotify={onNotify} />;
}
```

New idea. `Routes` holds the list of rules. Each `Route` is one rule: *when the address is `path`, show `element`*.

```

<Routes>
  <Route path="/" element={<HomePage />} />
  <Route
    path="/community"
    element={<CommunityPage onNotify={addNotification} />}
  />
  <Route
    path="/opportunities"
    element={<OpportunitiesPage onNotify={addNotification} />}
  />

```

Line by line.

- `<Routes>` — the list of rules. Only the best matching rule is shown.
- `path="/community"` — the address this rule is about.
- `element={<CommunityPage onNotify={addNotification} />}` — what to show. Notice that `element` is ordinary JSX, so passing props is exactly like using any component. `addNotification` still lives in `App`: nothing about that pattern changed.
- `/` is short enough to stay on one line; `/community` and `/opportunities` are written on several lines only because they are long — your editor's formatter (Prettier) makes that call, not you.

Why do we need this? Before, `<main>` always drew both panels. Now it draws whatever the address chooses. Nothing is hard-coded any more.

Common mistake. Writing the component as a *child* of `Route` instead of using `element`.

```

// WRONG: a component as the child of Route
<Route path="/community"><CommunityPage /></Route>

```

React Router throws an error (`[CommunityPage] is not a <Route> component. All component children of <Routes> must be a <Route> or <React.Fragment>`). Always use `element={...}`.

What happens at an address with no rule? `Routes` shows nothing: an empty page. (Making that friendlier is the homework.)

6. `Link`: moving without reloading

Familiar idea. In HTML, `<a href="/community">` takes you to another page. The browser asks the server for a brand-new document, throws away the running app, and starts again. For a React app that is very wasteful: every piece of state is lost.

New idea. `Link` looks like a link but does something smarter: it changes the address *inside* the running app and lets the router swap the page. No reload, no lost state. On the Home page:

TSX

```
<Link to="/community" className="home-link-card">
  <span className="home-link-title">Community</span>
  <span className="home-link-description">
    See what trainees and alumni are posting.
  </span>
</Link>
```

Line by line. `to` is the destination address (it is `to`, not `href`). Everything inside the `Link` is what the user clicks. You can prove there is no reload: open the browser's Network tab and click the card — no new document is requested.

Common mistake. Using `<a href>` inside a React Router app: it works, but the page reloads.

TSX

```
// WRONG: this reloads the whole page
<a href="/community">Community</a>
```

7. `NavLink`: a link that knows where you are

A navigation bar has a special need: it should show which page you are on. Before routing, the app had no idea what "the current page" was. `NavLink` is a `Link` that also knows whether it points at the current address. Its `className` can be a function that receives `{ isActive }`:

TSX

```
function navLinkClassName({ isActive }: { isActive: boolean }) {
  return isActive ? "nav-link nav-link-active" : "nav-link";
}
```

TSX

```
<nav className="navbar-links">
  <NavLink to="/" end className={navLinkClassName}>
    Home
  </NavLink>
  <NavLink to="/community" className={navLinkClassName}>
    Community
  </NavLink>
</nav>
```

Why is this powerful? We do not store "which page is showing" in state. The address *is* that state, and the router computes `isActive` from it.

Why does Home have `end`? By default a `NavLink` is active when the address *starts with* its `to`. Every address starts with `/`, so a Home link without `end` would be active on every page (we saw this happen in class). `end` says: "only when the whole address matches". Community does not need it: no other page's address starts with `/community`.

8. The teamwork: building the opportunity detail page yourself

In the teamwork your team built a real page for one opportunity: a new page file in `src/pages/`, a dynamic `Route` for `/opportunities/:id`, and everything needed to read the id from the address, load and show the matching opportunity, and handle loading/error/not-found. Sections 9–12 below walk through exactly what that involved. If your team can explain each idea, you have understood the core of the session.

9. Your teamwork: a route with a variable: `/opportunities/:id`

The problem. There are 20 opportunities. We cannot write 20 routes, and the data can change. We want `/opportunities/101`, `/opportunities/205` and any other id to show one page.

New idea. A colon in a route's `path` marks a **variable part** of the address:

TSX

```
<Route
  path="/opportunities/:id"
  element={<OpportunityDetailPage onNotify={addNotification} />}
/>
```

`:id` matches any value in that position. One route, any id. Because the detail page has its own address, it can be bookmarked, shared, opened in a new tab and refreshed — none of which worked when "View details" just expanded a card.

10. Your teamwork: `useParams`, reading the address from a component

The route matched, but the component does not yet know *which* opportunity was asked for. `useParams()` returns the variable parts of the current address:

TSX

```
const { id } = useParams();
const navigate = useNavigate();
```

The name inside the braces (`id`) has to match the name after the colon in the route (`:id`).

Something to remember: an address is text. `id` is a string (or `undefined`), even when the address contains digits. Our opportunities have numeric ids. So to find the matching opportunity we convert the number to text before comparing:

TSX

```
try {
  const opportunities = await fetchOpportunities();
  const match = opportunities.find((el) => String(el.id) === id) ?? null;

  setOpportunity(match);
  setIsLoading(false);
} catch (error) {
  console.log(error);
  setHasError(true);
  setIsLoading(false);
}
```

Line by line.

- `fetchOpportunities()` — the function we already had. The server has no "one opportunity" endpoint yet (backend work comes later), so we fetch the list and pick one.
- `opportunities.find(...)` — the first opportunity for which the test is true, or `undefined`.
- `String(el.id) === id` — number turned into text, compared with the address's text.
- `?? null` — "if nothing matched, use `null`".
- `try / catch` — the loading-and-error pattern you already know, so that a failing server shows an error instead of hanging. The `console.log(error)` is a real, harmless debugging leftover from class — not something you need to add on purpose.

Common mistake. Comparing without converting.

TSX

```
// WRONG: number compared with string | undefined
const match = opportunities.find((el) => el.id === id);
```

TypeScript refuses this: This comparison appears to be unintentional because the types 'number' and 'string | undefined' have no overlap.

The effect depends on the address. We load again whenever `id` changes, so going from one detail page to another shows the new opportunity:

TSX

```
useEffect(() => {
  load();
}, [id]);
```

As actually delivered, this triggers one real ESLint warning (`react-hooks/exhaustive-deps`: `load` is used inside the effect but not listed in `[id]`) that was not fixed in class. It is harmless here — `load` does not change between renders in a way that matters — but it is a genuine, confirmed warning in the shipped code, not something to silently "clean up" in your own copy without understanding why it is there.

11. Your teamwork: three states and a missing opportunity

Like every page that loads data, the detail page handles *loading*, *error* and the case where the id does not match anything (someone typed a wrong address):

TSX

```
{!isLoading && !hasError && !opportunity && (  
  <div className="opportunity-detail-not-found">  
    <p>No opportunity found with id "{id}".</p>  
    <Link to="/opportunities" className="view-details-button">  
      Back to Opportunities  
    </Link>  
  </div>  
)}
```

Why? With addresses, anyone can type anything. A page that only handles the happy path shows a blank card for a wrong id. This one explains and offers a way out — with a `Link`.

12. Your teamwork: `useNavigate`, going back from a button

`Link` is for a link the user clicks. Sometimes we need to change the address from **inside a function**: for example a *Back button*. `useNavigate()` returns a function for that:

TSX

```
<button className="back-button" onClick={() => navigate(-1)}>  
  ← Back  
</button>
```

`navigate(-1)` means "go one step back in the browser's history" — the same as pressing the browser's Back button. Note `() => navigate(-1)`: we give the button a *function to run on click*. Writing `onClick={navigate(-1)}` would run it immediately, while rendering.

Link or navigate? Use `Link` when the destination is a specific address the user might open in a new tab. Use `navigate` when an action in code decides where to go, or when "back" is not a fixed address.

13. Why we could delete so much code

The old "View details" was a button that flipped an `expandedId` state, and we wrote a stack of previously viewed opportunities (`detailHistory`), a handler to go back, a lookup table and a whole component (`BackToPreviousOpportunity`) just to imitate browser history. Once "View details" became a `Link` to a real address:

```

<Link
  className="view-details-button"
  to={` /opportunities/${opportunity.id}`}
>
  View details
</Link>

```

the address and the browser's real history do all of that for free. That is why the code could go: not because it was bad, but because a better mechanism replaced it. We searched the project for the component's name first, to *prove* nothing used it, before deleting the file.

Glossary

Word	Meaning
URL / address	The text in the browser's address bar, for example <code>/community</code> .
Route	One rule: an address (<code>path</code>) and what to show for it (<code>element</code>).
Router	The part that watches the address and picks the matching route.
Page	A component that an address shows. Lives in <code>src/pages/</code> .
Section	A reusable piece of UI that a page may use.
Dynamic segment / param	A variable part of an address, written with a colon: <code>:id</code> .
Active link	A navigation link that points at the current page.
Programmatic navigation	Changing the address from code (<code>navigate</code>), not from a click on a link.
History	The list of addresses the browser remembers for Back and Forward.
Single-page app	An app that loads one HTML page and draws everything with JavaScript.

Review questions

1. What could you not do with the app before routing, and why?
2. What is the difference between a page and a section?
3. What does `element` receive, and how do you pass props to a page from a route?
4. What is the difference between `Link` and a normal `<a href>`?
5. Why does the Home `NavLink` need `end`, but the Community one does not?
6. What does `:id` mean in a route's `path`, and how does a component read it?
7. Why do we write `String(el.id) === id`?
8. When would you use `navigate(-1)` instead of a `Link`?
9. Why was it safe to delete the old history stack code?

Answers

1. You could not share a link to a specific page, refresh and stay where you were, or use the browser's Back button: the whole app was one page with one address. We even faked history by hand.
2. A page is a component that an address shows (it lives in `src/pages/`). A section is a reusable piece of UI that a page (or another component) uses. `CommunityPage` shows `CommunitySection`.
3. `element` receives JSX, so props are passed exactly as with any component: `element={<CommunityPage onNotify={addNotification} />}`.
4. A normal `<a href>` asks the server for a new document and restarts the app, losing state. `Link` changes the address inside the running app, so nothing reloads.
5. `NavLink` is active when the address starts with its `to`. Every address starts with `/`, so Home would be active everywhere. `end` makes it match only exactly. `/community` is not the start of any other page.
6. `:id` marks a variable part of the address. In the component, `useParams()` returns it, for example `const { id } = useParams();`.
7. An address is text, but opportunity ids are numbers. Converting the number to text lets the two be compared (and TypeScript refuses to compare a number with `string | undefined`).
8. `navigate(-1)` for an action in code, such as a Back button that returns to the previous entry in the browser's history. `Link` when the destination is a specific address the user might click or open in a new tab.
9. Because the address and the browser's history now do the job that code was written to fake. We checked that nothing used the component before deleting it.

What you should be able to explain now

- ☐ Why a single-page app needs routing, in one or two sentences.
- ☐ What `BrowserRouter`, `Routes`, `Route`, `Link` and `NavLink` each do.
- ☐ What a page is, and how to add one (file + `Route` + `NavLink`).
- ☐ Why `<a href>` reloads the page and `Link` does not.
- ☐ Why Home needs `end`.
- ☐ What `:id`, `useParams` and `useNavigate` do, and why an address value is text.
- ☐ Why routing let us delete code.

Not yet

These are natural next questions, but we have **not** covered them yet, so do not use them in your work: nested routes and shared layouts, redirects, protected pages (pages only for logged-in users), reading the query string of an address, the "current location" object, and the newer data-loading style of routing. Also not covered: what to show for an address that matches no route (that is the homework), Context, Redux, and the Express/Node backend topics. They come later in the Bootcamp.